

```
badguys=[[640,100]]
healthvalue=194
```

This sets up a timer (as well as a few other values) so that the game adds new moles after some time has elapsed. Decrease all badtimer frames until these reach zero to release a new mole.

Add the following to the end of section #3:

```
badguyimg1 = pygame.image.load("resources/images/badguy.png")
badguyimg=badguyimg1
```

The first line above is similar to all the previous image-loading codes. The second line sets up a copy of the image so that you can animate the bad guy much more easily.

Update and show the bad guys. Add the code given below after section #6.2:

```
# 6.3 - Draw moles
if badtimer==0:
    badguys.append([640, random.randint(50,430)])
    badtimer=100-(badtimer*2)
    if badtimer1>=35:
        badtimer1=35
    else:
        badtimer1+=5

index=0
for badguy in badguys:
    if badguy[0]<=64:
        badguys.pop(index)
        badguy[0]=7
        index+=1
for badguy in badguys:
    screen.blit(badguyimg, badguy)
```

The first line checks if badtimer is zero, and if it is, creates a mole and sets up badtimer again based on the number of times badtimer has run so far. The first 'for' loop updates x position of the moles, checks if moles are off screen and removes these if so.

The second 'for' loop draws all moles. In order to use the random function in the above code, import the random library. Add the following code to the end of section #1:

```
import random
```

Add the following after the while

statement (section #4) to decrement the value of badtimer for each frame:

```
badtimer-=1
```

Run the program. Now you would start seeing some real game-play—you can let the bunny shoot, move and turn, while the moles try to turn towards him (Fig. 5).

But wait! Why are the moles not blowing up the castle? Add the code given below before index += 1 on the first for loop in section #6.3:

```
# 6.3.1 - Attack castle
badrect=pygame.Rect(badguyimg.get_rect())
badrect.top=badguy[1]
badrect.left=badguy[0]
if badrect.left<64:
```



Fig. 4: Shooting with an arrow

```
healthvalue -= random.randint(5,20)
badguys.pop(index)
# 6.3.3 - Next bad guy
```

If the moles' x value is less than 64 to the right, then delete that bad guy and decrease the game health value by a random value between 5 and 20.

If you build and run the program, you would get a bunch of attacking moles who vanish when they hit the castle. Although you cannot see it, the moles are actually lowering your health.

Collisions with moles and arrows

The moles are attacking your castle. It is time to set the arrows to kill the moles so you can safeguard your castle and win the game. Basically, loop through all the bad guys and inside each of those loops, loop through all the arrows and check if these collide. If they do, then delete

the moles and the arrow, and add one to your accuracy ratio.

Right after section #6.3.1, add the following:

```
#6.3.2 - Check for collisions
index1=0
for bullet in arrows:
    bullrect=pygame.Rect(arrow.get_rect())
    bullrect.left=bullet[1]
    bullrect.top=bullet[2]
    if badrect.colliderect(bullrect):
        acc[0]+=1
        badguys.pop(index)
        arrows.pop(index1)
        index1+=1
```

If you run the program, you should finally be able to shoot and kill the moles.

Add a HUD with health meter and clock. Now all you need is a way to keep score and to show how well the bunny is doing. The easiest way to do this is to add a heads-up display (HUD) that shows the current health level of the castle. You can also add a clock to show how long the castle survived.

Add the clock and add the following code right before the beginning of section #7:

```
# 6.4 - Draw clock
font = pygame.font.Font(None, 24)
survivedtext = font.render(str((90000-pygame.time.get_ticks())/60000)+"str((90000-pygame.time.get_ticks())/1000*60).zfill(2), True, (0,0,0))
```

Next, add the health bar. But before drawing that, load the images for the bar. Add the following code to the end of section #3:

```
healthbar = pygame.image.load("resources/images/healthbar.png")
health = pygame.image.load("resources/images/health.png")
```

The first is the red image used for the full health bar. The second is the green image used to show the current health level. Add the following code right after section #6.4 (that you just added) to draw the health bar on the screen:

```
# 6.5 - Draw health bar
screen.blit(healthbar, (5,5))
```